UNIT-3
CHAPTER II: RELATIONAL DB DESIGN

FUNCTIONAL DEPENDENCIES

Definition:

A functional dependency, denoted by **X → Y**, between two sets of attributes X and Y that are subsets of R specifies a constraint on the tuples of the relation table i.e r of R. The constraint is that, for any two tuples t1 and t2 in r that have **t1[X] = t2[X]**, they must also have **t1[Y] = t2[Y]**.
In more detailed way:

- If **t1** and **t2** are two rows in the table **r** (an instance of relation R), and they have the **same values for X**, then they must also have the **same values for Y**.
- Mathematically:
  If **t1[X] = t2[X]**, then **t1[Y] = t2[Y]**.

Consider a **Student table** with attributes:

- **Student_ID** (unique ID of a student)
- **Student_Name** (name of the student)
- **Course** (course the student is enrolled in)

Then, if two rows have the **same** Student_ID, they **must** have the same Student_Name. Because Student_ID uniquely identifies a student, a particular Student_ID cannot have multiple different Student_Name values.

Inference Rules in Functional Dependency:

Inference rules are logical rules used to derive new functional dependencies (FDs) from given FDs in a **relational database**. These rules help in determining **all possible FDs** that hold in a relation.

Armstrong's Axioms define a set of **sound** (correct) and **complete** (able to derive all valid FDs) rules for inferring FDs. First three rules **Reflexivity, Transitivity and Augmentation** are Armstrong's Axioms.

IR1 (reflexive rule):

- If **X ⊇ Y**, then **X →Y.**
- The **Reflexivity Rule** in functional dependency states that if **X** is a set of attributes and **Y** is a subset of **X** (denoted as **X ⊇ Y**), then **X functionally determines Y** (written as **X → Y**). This means that if we know the values of all attributes in **X**, we automatically know the values of all attributes in **Y**, since **Y** is already included within **X**.
- For example, In a **Student** table with attributes **{Roll_Number, Name, Course, Year}**, since Roll_Number is a subset of **{Roll_Number, Name}**, the Reflexivity Rule states that **{Roll_Number, Name} → {Roll_Number}**, meaning if we know both Roll_Number and Name, we automatically know Roll_Number.

IR2 (augmentation rule):

- **{X → Y} | = XZ → YZ.**
- The Augmentation Rule states that if a functional dependency **X → Y** holds, then adding the same set of attributes **Z** to both sides does not change the dependency, meaning **XZ → YZ** also holds.
- For example, consider a Student table with attributes **{Roll_Number, Name, Course, Year}**. Suppose we have the functional dependency **{Roll_Number} → {Name}**, which means that each student's roll number uniquely determines their name. By the Augmentation Rule, if we add Course to both sides, we get **{Roll_Number, Course} → {Name, Course}**, meaning that if we know both Roll_Number and Course, we can still determine Name along with Course, which was already known. This rule ensures that adding redundant attributes does not alter the fundamental dependency relationships in a database.

IR3 (transitive rule):

- **{X → Y, Y → Z} |= X → Z.**
- The Transitivity Rule states that if **X → Y and Y → Z** hold in a relation, then we can infer that **X → Z**. This means that if X determines Y, and Y further determines Z, then X must also determine Z indirectly.
- For example, in a **Student** table with attributes **{Roll_Number, Name, Department}**, suppose we have the functional dependencies **{Roll_Number} → {Name}** and **{Name} → {Department}.** By applying the **Transitivity Rule**, we can conclude that **{Roll_Number} → {Department}**, meaning that if we know a student's roll number, we can determine their department, even though this was not explicitly stated in the original dependencies. This rule is useful in database design as it helps identify indirect dependencies and ensures proper normalization of relations.

IR4 (decomposition or projective rule):

- **{X → YZ} |= X → Y.**
- The Decomposition Rule states that if **X → YZ**, then **X → Y** must also hold. This means that if X determines a set of attributes YZ, then X must determine each attribute in that set individually.
- For example, consider a **Student** table with attributes **{Roll_Number, Name, Course}**. Suppose we have the functional dependency **{Roll_Number} → {Name, Course}**. By applying the **Decomposition Rule**, we can infer two separate dependencies: **{Roll_Number} → {Name}** and **{Roll_Number} → {Course}**. This rule is useful in database normalization as it allows breaking down complex dependencies into simpler ones, ensuring better organization and reducing redundancy in relational schema design.

IR5 (union or additive rule):

- **{X → Y, X → Z} |= X → YZ.**
- The Union Rule states that if **X → Y** and **X → Z** hold, then we can infer **X → YZ**. This means that if X determines Y and X separately determines Z, then X must also determine the combination of both Y and Z together.

- For example, consider a **Student** table with attributes **{Roll_Number, Name, Course}**. Suppose we have the functional dependencies **{Roll_Number} → {Name}** and **{Roll_Number} → {Course}**. By applying the **Union Rule**, we can infer **{Roll_Number} → {Name, Course}**, meaning that knowing a student's Roll_Number allows us to determine both their Name and Course together. This rule is useful in database design as it helps in merging dependencies and maintaining consistency while ensuring data integrity.

IR6 (pseudo-transitivity rule):

- **{X → Y, WY → Z} |= WX → Z.**
- The Pseudotransitivity Rule in functional dependency states that if **X → Y** and **WY → Z**, then we can infer **WX → Z.** This means that if X determines Y, and when combined with W, Y determines Z, then WX must determine Z.
- For example, consider a Student table with attributes **{Roll_Number, Department, Course, Fees}**. Suppose we have the following functional dependencies:

1. **{Roll_Number} → {Department}.**
2. **{Department, Course} → {Fees}.**

By applying the Pseudotransitivity Rule, we can infer: **{Roll_Number,Course}→Fees.** This rule is useful in database normalization as it helps in simplifying and restructuring dependencies for an efficient relational schema.

Trivial Dependency

- A **functional dependency (FD) X → Y** is **trivial** if **Y is a subset of X (X ⊇ Y)**, meaning that **X already contains Y**.
- Consider a **Student** table with attributes **{Roll_Number, Name, Course}**.
- If we have the dependency **{Roll_Number, Name} → {Roll_Number}**, it is **trivial** because **{Roll_Number}** is already a part of **{Roll_Number, Name}.**
- Reflexivity is trivial.

Non-Trivial Dependency

- A **functional dependency X → Y** is **non-trivial** if **Y is not a subset of X (X ⊉ Y)**, meaning **Y contains at least one attribute that is not already in X**.
- Example: **{Roll_Number} → {Name}** is **non-trivial** because Name is not a subset of **{Roll_Number},** it is a separate attribute being determined by Roll_Number.
- All others except for reflexivity are non-trivial.

Key Difference:

- **Trivial dependency**: Provides no new information (X ⊇ Y).
- **Non-trivial dependency**: Provides meaningful constraints in a relation (X ⊉ Y).

Closure: (F^+)

- **Closure F^+ of F** is the **set of all functional dependencies** that can be inferred from a given set **F**. It helps in **finding all dependencies** that hold in a relation.
- Inference rules **help systematically derive new functional dependencies** from existing ones.
- They ensure that no important dependencies are **missed** in database schema design.
- **Ensures Soundness:** If we infer a functional dependency **X → Y** from **F**, then it must **always hold** in every possible relation state. Ensures that no **incorrect** dependencies are inferred.
- **Ensures Completeness:** By applying **IR1 to IR3 repeatedly**, we can **derive all possible** functional dependencies. Ensures that the **full closure (F^+) is obtained**.
- **<u>Why Is Closure (F^+) Important?</u>**

1. **Used in database normalization** to remove redundancy.
2. **Helps in finding candidate keys** by identifying all dependencies.
3. **Ensures consistency** in a relational database schema.

Algorithm 16.1. Determining X^+, the Closure of X under F

Input: A set F of FDs on a relation schema R, and a set of attributes X, which is a subset of R.

$X^+ := X$;
repeat
    old$X^+ := X^+$;
    for each functional dependency $Y \rightarrow Z$ in F do
        if $X^+ \supseteq Y$ then $X^+ := X^+ \cup Z$;
until (X^+ = oldX^+);

This algorithm **determines the closure (X^+) of a set of attributes X** under a given set of **functional dependencies (F)**. The closure X^+ represents all attributes that can be derived from **X** using the functional dependencies in **F**.

1. **Initialize X^+ with X:**
    - Start with $X^+ = X$ (the closure initially contains only the attributes in X).
2. **Iterate Until No More Changes:**
    - Keep track of the old value of X^+ (store it as oldX^+).
    - Loop through **each functional dependency (Y → Z) in F**:
        - If **Y is already in X^+** (i.e., $Y \subseteq X^+$), then add **Z** to X^+.
    - Repeat the process until **X^+ no longer changes** (i.e., X^+ = oldX^+).

<u>Example:</u> R: {A, B, C, D, E} ; FDs: {A → B, B → C, AC → D, D → E}

1. **Start:** $X^+ = \{A\}$
2. **Apply A → B** → $X^+ = \{A, B\}$
3. **Apply B → C** → $X^+ = \{A, B, C\}$
4. **AC → D** (since A and C are in X^+) → $X^+ = \{A, B, C, D\}$
5. **D → E** → $X^+ = \{A, B, C, D, E\}$

6. **No more changes** → Final closure: **X^+ = {A, B, C, D, E}**

<u>Example:</u> FDs: {Ssn →Ename, Pnumber → {Pname, Plocation}, {Ssn, Pnumber} → Hours}

1. <u>Finding Closure of {Ssn}$^+$</u>

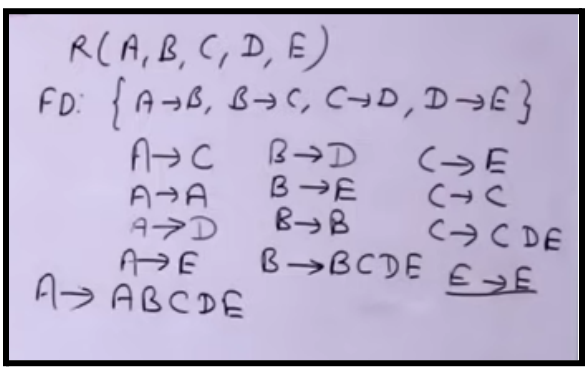
{Ssn}$^+$ = {Ssn}
{Ssn}$^+$ = {Ssn, Ename}
Final Closure: {Ssn}$^+$ = {Ssn, Ename}

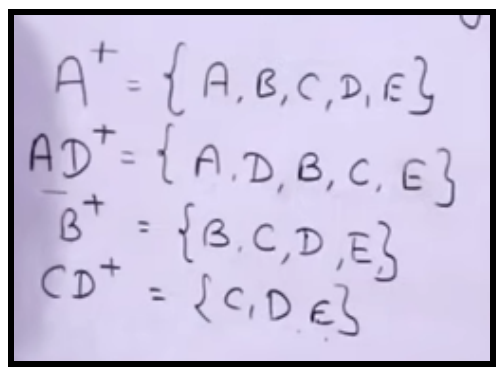
2. <u>Finding Closure of {Pnumber}$^+$</u>

{Pnumber}$^+$ = {Pnumber}
{Pnumber}$^+$ = {Pnumber, Pname, Plocation}
Final Closure: {Pnumber}$^+$ = {Pnumber, Pname, Plocation}

3. <u>Finding Closure of {Ssn, Pnumber}$^+$</u>

{Ssn, Pnumber}$^+$ = {Ssn, Pnumber}
{Ssn, Pnumber}$^+$ = {Ssn, Pnumber, Ename}
{Ssn, Pnumber}$^+$ = {Ssn, Pnumber, Ename, Pname, Plocation}
{Ssn, Pnumber}$^+$ = {Ssn, Pnumber, Ename, Pname, Plocation, Hours}
Final Closure: {Ssn, Pnumber}$^+$ = {Ssn, Pnumber, Ename, Pname, Plocation, Hours}





<u>Key:</u>

Note: Superkey is the set of attbs whose closure contains all the attbs present in R. In the above example, A, AD are superkeys.

Algorithm 16.2(a). Finding a Key *K* for *R* Given a set *F* of Functional Dependencies

Input: A relation *R* and a set of functional dependencies *F* on the attributes of *R*.

1. Set *K* := *R*.
2. For each attribute *A* in *K*
   {compute $(K - A)^+$ with respect to *F*;
   if $(K - A)^+$ contains all the attributes in R, then set K := K – {A} };

This algorithm **finds a candidate key (K) for a relation R**, given a **set of functional dependencies (F).** A **candidate key** is a minimal set of attributes that can **uniquely**

identify all attributes in the relation.

1. **Start with K = R**
   - Assume that the **entire set of attributes (R)** is a superkey.
   - This means that all attributes together can uniquely determine the relation.
2. **Try to Minimize K (Remove Unnecessary Attributes)**
   - For each attribute **A** in **K**:
     - Compute the **closure of (K - A)**$^+$ with respect to **F** (this means finding the attributes that can be derived from K after removing A).
     - If **(K - A)**$^+$ **still includes all attributes in R**, then **A is not needed** and can be removed from K.
     - Update **K := K - {A}**.
3. **End Condition:**
   - When we cannot remove any more attributes while still being able to derive all attributes in R, the remaining **K** is a **candidate key**.

<u>Example</u>

R={Ssn, Ename, Pnumber, Pname, Plocation, Hours}

FDs= {Ssn → Ename, Pnumber → {Pname, Plocation}, {Ssn, Pnumber} → Hours}

<u>Step 1: Start with K = R</u>

K={Ssn, Ename, Pnumber, Pname, Plocation, Hours}

<u>Step 2: Try Removing Attributes One by One</u>

- Remove **Ename**: Compute {Ssn, Pnumber, Pname, Plocation, Hours}$^+$
  - {Ssn} → {Ssn, Ename} (so we can still determine Ename)
  - {Ssn, Pnumber} → Hours (remains valid)
  - {Pnumber} → {Pname, Plocation} (remains valid)
  - **Closure contains all attributes, so remove Ename**

  K={Ssn, Pnumber, Pname, Plocation, Hours}

- Remove **Pname and Plocation**: Compute {Ssn, Pnumber, Hours}$^+$
  - {Ssn} → {Ename} (remains valid)
  - {Pnumber} → {Pname, Plocation} (remains valid)
  - {Ssn, Pnumber} → Hours (remains valid)
  - **Closure still contains all attributes, so remove Pname and Plocation**

  New K={Ssn, Pnumber, Hours}

- Remove **Hours:** Compute {Ssn, Pnumber}$^+$
  - {Ssn} → {Ename} (remains valid) and {Pnumber} → {Pname, Plocation}

- {Ssn, Pnumber} → Hours (remains valid). **New K= {Ssn, Pnumber}**

- **Candidate Key = {Ssn, Pnumber}**
- This means **Ssn and Pnumber together uniquely determine all attributes in the relation**.

Q) finding key in $\{B \to D, D \to A\}$ above ques:

Step (1) ⇒ $R = \{A, B, D\}$

$K = \{A, B, D\}$

Step (2) ⇒ remove A : $B^+ = BDA$ $D^+ = AD$ (all attrs can be derived w/o A)

remove B : $A^+ = A$ $D^+ = DA$ (not all attrs ~~~ w/o B)

remove D : $A^+ = A$ $B^+ = BDA$ (all attrs can be derived w/o D)

∴ $K = \{B\}$ // ∴ B is the key as it's min set of attrs that can derive all other attrs //

Equivalence of FDs:

Definition: A set of functional dependencies **F covers E** if every functional dependency in **E** can be **derived from F**$^+$ (the closure of F).

- **Mathematically:** E ⊆ F+
- This means every functional dependency in **E** can be inferred from **F**.

Definition: Two sets of functional dependencies **E and F are equivalent** if they determine the **same set of dependencies**.

- This means that the closure of **E** (E$^+$) is the same as the closure of **F** (F$^+$).
- **Mathematically:** E+ = F+
- This is **only true if**:
  - **F covers E** → Everything in E can be derived from F.
  - **E covers F** → Everything in F can be derived from E.

* How to check if F covers E? ($F \supseteq E$)

i) Take each FD $X \to Y$ that exists in 'E'.
ii) Cal X^+ using rules in F.
iii) Check if Y's attrs are included in X^+.
iv) if it works for every single FD in E, F covers E.

* are E & F equivalent?

i) F covers E ($F \supseteq E$)
ii) E covers F ($E \supseteq F$)
iii) If both are true, they are equivalent

Q) $F = \{A \to B, B \to C\}$

$E = \{A \to B, B \to C, A \to C\}$

(i) F covers E? ⇒ Closure of 1st FD of 'E' i.e (A^+) ⇒ $A^+ = ABC$ (from rules in 'F') (check acc to 'E' if present)

next ⇒ $B^+ = BC$

(ii) E covers F? ⇒ Closure of 1st FD of 'F' i.e (A^+) ⇒ $A^+ = ABC$

⇒ $B^+ = BC$

(iii) Since $F \supseteq E$ and $F \subseteq E$, $F \equiv E$ // equivalent //

Q) $X = \{AB \to CD,\ B \to C,\ C \to D\}$
$Y = \{AB \to C,\ AB \to D,\ C \to D\}$

(i) X covers Y? $\Rightarrow AB^+ = ABCD$
$\Rightarrow C^+ = CD$

(ii) Y covers X? $\Rightarrow AB^+ = ABCD$
$\Rightarrow B^+ = B$ ×
$\Rightarrow C^+ = CD$

(iii) $X \not\equiv Y$ //

Minimal Cover of FDs:

Definition: A minimal cover of a set of functional dependencies E is a minimal set of dependencies (in the standard canonical form and without redundancy) that is equivalent to E. We can always find at least one minimal cover F for any set of dependencies E.

A **minimal cover** (also called **canonical cover**) is the **simplest** and most **efficient** representation of a given set of functional dependencies **E**, without any unnecessary redundancy. It gives the standardized or canonical form.

A **minimal cover F** for a set of functional dependencies **E** must satisfy the following conditions:

1. **F is equivalent to E ($F^+ = E^+$)**
   - This means **F and E** must define the **same** functional dependencies.
2. **All dependencies in F are in standard form**
   - Each FD must have **only one attribute** on the **right-hand side**.
   - Example: ***{A → BC}*** must be split into ***{A → B, A → C}.***
3. **F has no redundant functional dependencies**
   - Removing any FD from **F** should make it **not equivalent** to **E**.
   - Example: If ***{A → B, B → C, A → C}*** is given, then ***{A → C}*** is **redundant** because we can derive ***A → C*** using ***A → B*** and ***B → C*** **(transitivity)**.
4. **F has no redundant attributes**
   - If an attribute can be removed from the left-hand side without changing F^+, then it is redundant.
   - Example: ***{AB → C, A → C}*** → The second FD ***{A → C}*** makes ***{AB → C}*** **redundant** since ***A → C*** already exists.

Algorithm 16.2. Finding a Minimal Cover F for a Set of Functional Dependencies E

Input: A set of functional dependencies E.

1. Set $F := E$.
2. Replace each functional dependency $X \to \{A_1, A_2, \ldots, A_n\}$ in F by the n functional dependencies $X \to A_1, X \to A_2, \ldots, X \to A_n$.
3. For each functional dependency $X \to A$ in F
   for each attribute B that is an element of X
   if $\{ \{F - \{X \to A\}\} \cup \{ (X - \{B\}) \to A\} \}$ is equivalent to F
   then replace $X \to A$ with $(X - \{B\}) \to A$ in F.
4. For each remaining functional dependency $X \to A$ in F
   if $\{F - \{X \to A\}\}$ is equivalent to F,
   then remove $X \to A$ from F.

Q) Find Minimal Cover for the F.D: $\{A \to B, C \to B, D \to ABC, AC \to D\}$.

Step①

⟹ RHS should have only 1 attr. Separate composite

$\{A \to B \quad C \to B \quad D \to A \quad D \to B \quad D \to C \quad AC \to D\}$

Step② ⇒ remove redundant FDs.

$A^+ = A \quad C^+ \to C \quad D^+ = DBC \quad D^+ = DCBA \quad D^+ = DAB \quad AC^+ = ACB$

$A \to B \quad C \to B \quad D \to A \quad D \to B \quad D \to C \quad AC \to D \implies \{A \to B, C \to B, D \to A, D \to C, AC \to D\}$

(say, we remove $A \to B$, check if we can get A^+ from other dependencies in the FD, excluding $A \to B$) (since we do not have $A \to B$, we cannot remove it, not redundant)

(we can remove it)

Step③: LHS should have only one attr:

$A \to B \quad C \to B \quad D \to A \quad D \to C \quad AC \to D$

→ how to make it single?

(If in $AC \to D$, we remove 'A', & if you take C^+ and get A in C^+, we can remove 'A')

$C^+ = C$ (check in the above set) (no 'A') (we cannot remove 'A').

(now do vice-versa)

$A^+ = A$ (no 'C', we cannot remove 'C' as well).

∴ we cannot further reduce step②.

Q) $E: \{B \to A, D \to A, AB \to D\}$, find Min Cover?

⟹ (i) $\{B \to A \quad D \to A \quad AB \to D\}$

⟹ (ii) $\{B \to A \quad D \to A \quad AB \to D\}$

(∵ excluding AB) $B^+ = BD \quad D^+ = D \quad AB^+ \to AB$

⟹ (iii) $\{B \to A \quad D \to A \quad AB \to D\}$

$AB \to D \implies B^+ = BA$ (can remove A)

∴ $B \to D$ can replace $AB \to D$

∴ $\{B \to A, D \to A, B \to D\}$ Ans: $\{B \to D, D \to A\}$

(iv) Step② repeat ⇒ $B \to A \quad D \to A \quad B \to D$

$B^+ = BDA \quad D^+ = D \quad B^+ = BA$

∴ Min Cover = $\{D \to A, B \to D\}$

Why Do We Need a Minimal Cover?

- **Removes unnecessary dependencies** to simplify the database schema.
- **Helps in database normalization** (e.g., BCNF, 3NF).
- **Reduces redundancy**, improving efficiency in query processing.
- **Ensures a minimal yet complete set of constraints** in a relational schema.

NORMAL FORMS BASED ON PRIMARY KEYS

Normalization is a **formal process** used to **test and improve relational schemas** by reducing redundancy and avoiding anomalies (insertion, deletion, update issues). It was proposed by **Edgar F. Codd in 1972** and follows a **top-down approach**, evaluating each relation against normal form criteria and decomposing relations if necessary.

Normalization

- A step-by-step process that **improves database design**.
- Based on **functional dependencies (FDs)** and **primary keys**.
- Reduces **data redundancy** and prevents **anomalies**.
- Uses a **series of normal forms (1NF, 2NF, 3NF, BCNF, 4NF, 5NF)**.

Types of Normal Forms Based on Functional Dependencies

Codd originally proposed **three normal forms** based on **functional dependencies**:

1. **First Normal Form (1NF)**
2. **Second Normal Form (2NF)**
3. **Third Normal Form (3NF)** (also known as BCNF in stricter cases)

These normal forms ensure that a database is **structured and avoids redundancy** by considering **functional dependencies** among attributes.

Higher Normal Forms Based on Multivalued and Join Dependencies

Later, **4NF and 5NF** were introduced, which deal with **multivalued dependencies** and **join dependencies** rather than just functional dependencies.

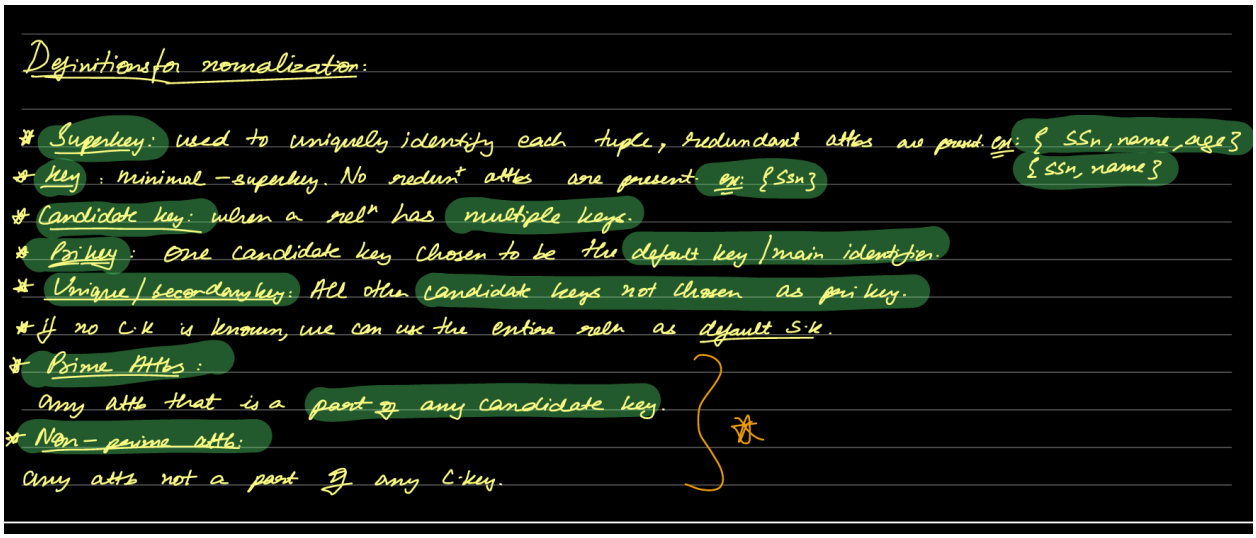
- **4NF (Fourth Normal Form)**: Deals with **multivalued dependencies** (MVDs).
- **5NF (Fifth Normal Form or PJNF - Project-Join Normal Form)**: Deals with **join dependencies** (JD).

These higher normal forms are used for **more advanced database optimization**.

Goals of Normalization

Normalization ensures the database design achieves:

1. **Minimizing Redundancy**
   - Avoids **data duplication**, reducing storage space and improving consistency.
2. **Minimizing Anomalies**
   - **Insertion anomaly**: Avoids unnecessary duplicate data when inserting a new record.
   - **Deletion anomaly**: Ensures important data isn't lost when deleting a record.
   - **Update anomaly**: Prevents inconsistent data updates.

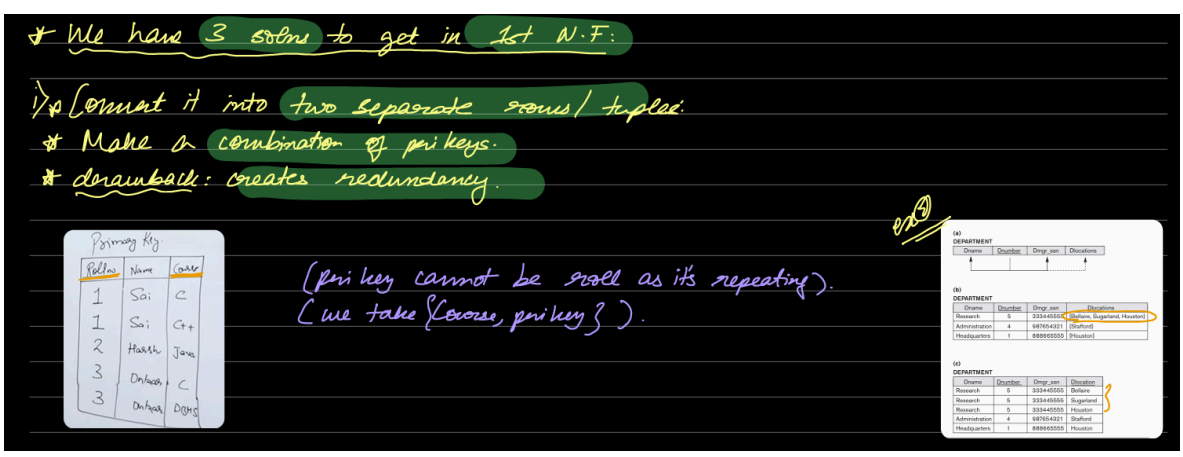


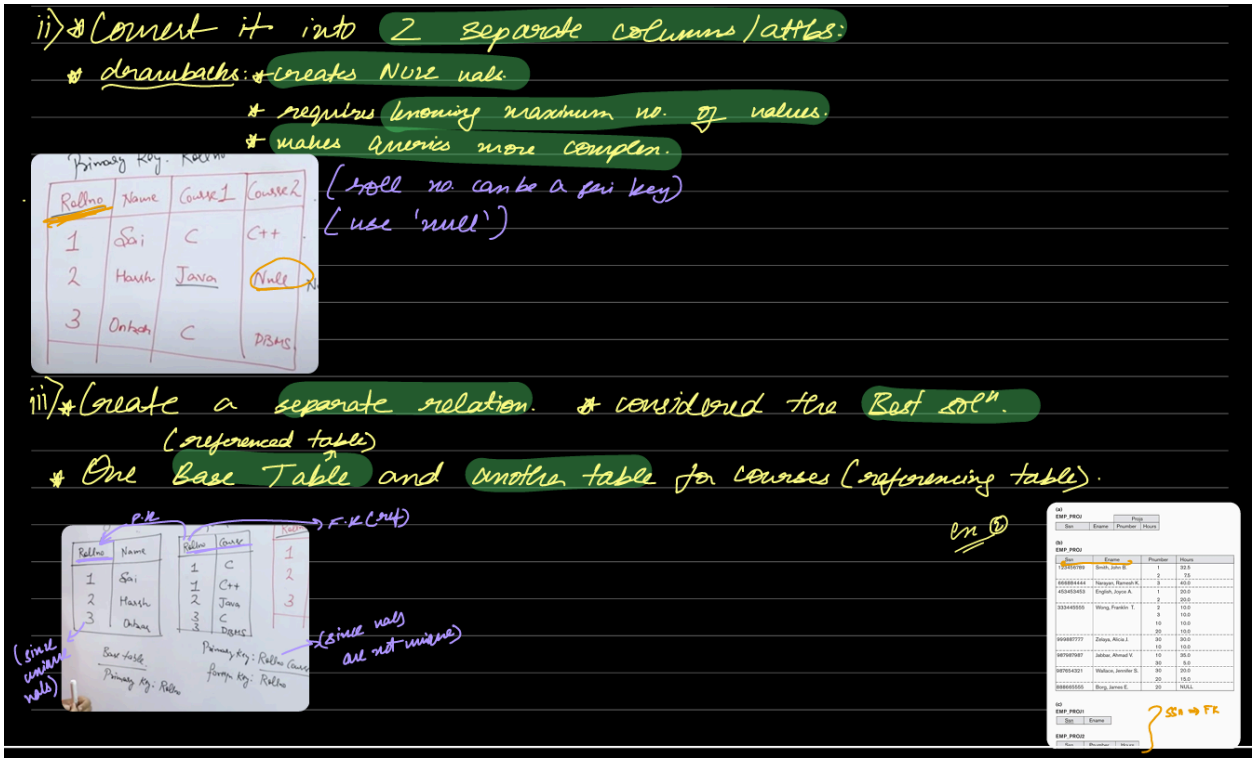
First Normal Form (1NF) :

First Normal Form (1NF) is the **basic requirement** for a relation in the **relational database model**. It ensures that data is stored in a **flat, tabular structure** by **eliminating multivalued and composite attributes**.

Rules of 1NF:

1. **Atomicity**: Each attribute must hold **atomic (indivisible) values**.
   - ❌ No lists, sets, or arrays in a single attribute.
   - ❌ No nested records or **relations within relations**.
2. **Unique Rows (Tuples)**: Each row must be **uniquely identifiable**, typically using a **primary key**.





Second Normal Form (2NF) :

Second Normal Form (2NF) is a **stronger version of 1NF**, ensuring that **all non-prime attributes** are **fully functionally dependent** on the **candidate key**.

Full vs. Partial Dependency

- **Full Functional Dependency:**
  - A functional dependency ***X → Y*** is **fully dependent** if removing **any attribute** from X **breaks** the dependency.
  - Example: ***{Ssn, Pnumber} → Hours***
    - Neither ***Ssn → Hours*** nor ***Pnumber → Hours*** hold individually.
    - **This is a full dependency**.
- **Partial Functional Dependency:**
  - A functional dependency ***X → Y*** is **partial** if **some attributes** in X can be removed, yet the dependency still holds.
  - Example: ***{Ssn, Pnumber} → Ename***
    - Since ***Ssn → Ename*** holds independently, the dependency is **partial**.

Note: 2NF removes partial dependencies.

Rules of 2NF: A relation is in 2NF if:

- It is **already in 1NF**.
- **Every non-prime attribute is fully functionally dependent** on the **candidate key**.
- **No partial dependencies exist** (i.e., no attribute should be dependent on part of a composite primary key).

* How to find C.keys in a Table? ⟹ Closure Method

Q) $R = (ABCDE)$

$FD = \{A \to B,\ BC \to D,\ E \to C,\ D \to A\}$ ('E' not present on right side so take E^+ first)

$\Rightarrow E^+ = EC$

$\Rightarrow (AE)^+ = ABECD$ $\therefore CK = \{AE\}$

⇒ Wkt: (Others?? BE^+, CE^+, DE^+ .... but time consuming) use this method instead

$CK = \{AE\}$ (check if A,E are present in R.H.S)

(replace with that attr) $DE^+ = DEABC$

$BE^+ = BECDA$ ✓ (if we replace B with A, same C.K will arise, STOP process).
$CE^+ = CE$ ✗

$\therefore CK = \{AE, DE, BE\}$

(or)

$ABCDE^+ = ABCDE$ (superkey)

$CK = \{E\}$

Q) $\{A \to B,\ D \to E\}$, $R(A,B,C,D,E)$.

$\Rightarrow ABCDE^+ = \{A,B,C,D,E\}$

$\Rightarrow ACDE^+ = \{A,B,C,D,E\}$

$\Rightarrow ACD^+ = \{A,C,D,B,E\}$

(we cant discard as nothing on RHS).

* Check if prime attrs are present in RHS of F.D, if no, it is a C.key.

$\therefore CK = \{A\ C\ D\}$ (not on RHS) (if not, find fronting find + of subsets)

Q) $\{A \to B,\ B \to C,\ C \to A,\ A \to C\}$, $R = (A,B,C,D)$

$\Rightarrow ABCD^+ = \{A,B,C,D\}$ (S.key)

$ACD^+ = \{A,B,C,D\}$ (S.key)

(cannot discard. as it is doesn't then?? B, 'C' is discarded) $AD^+ = \{A,B,C,D\}$ (S.key)

↳ is it C.key?? ⇒ Prime attrs: $\{A, D\}$ X C.key ⟹ now take subsets.

$\{A\}^+ = \{A,B,C\}$ X

$\{D\}^+ = \{D\}$ X

AD
↳ CD

$CK \Rightarrow \{AD, BD, CD\}$

* How to check if a F.D is in 2NF or not?

ex(1) FD: $\{A \to B, B \to C, C \to D, D \to E\}$

$R = (A\,B\,C\,D\,E\,F)$.

2NF (i) ⇒ it is in 1NF, yes.

C.key (ii) ⇒ $(AF)^+ = ABCDE$ ✓

$\therefore$ C.K = $\{A, F\}$

no PD (iii) ⇒ prime attrs = A, F

⇒ non prime attrs = B, C, D, E

⇒ for 2NF form, (proper subset of C.K $\xrightarrow{det}$ non prime attr) → Partial dependency.

in $\{A, F\}$ subsets are A ⇒ dets B (non prime) $\therefore$ Partial dep exists $\therefore$ not in 2NF //

ex(2) FD = $\{AB \to CD, C \to A, D \to B\}$

R (A, B, C, D).

(i) ⇒ 1NF

(ii) ⇒ Since all are in RHS: (use the other method)

(iii) ⇒ $AB^+ = \{A\,B\,C\,D\}$

$AB^+ = \{A\,B\,C\,D\}$

$CB^+ = CBAD$

$CD^+ = CDBA$

} normal method

C.K = $\{AB, CB, CD\}$

⇒ P.Attr = A B C D

⇒ non-prime = X (no non primes only) $\therefore$ 2NF form //

4 Example of a Relation That is NOT in 2NF

Consider the **EMP_PROJ** table:

Ssn	Pnumber	Ename	Pname	Plocation	Hours
123	1	Alice	ProjectA	NY	30
123	2	Alice	ProjectB	LA	40
456	2	Bob	ProjectB	LA	20

Functional Dependencies (FDs):

1. `{Ssn, Pnumber} → Hours` (Full Dependency) ✅
2. `Ssn → Ename` (Partial Dependency) ❌
3. `Pnumber → {Pname, Plocation}` (Partial Dependency) ❌

Why is EMP_PROJ Not in 2NF?

- The **primary key** is `{Ssn, Pnumber}`.
- **Ename is dependent only on** `Ssn`, not on the entire `{Ssn, Pnumber}`.
- **Pname and Plocation are dependent only on** `Pnumber`, not on `{Ssn, Pnumber}`.
- Since **partial dependencies exist**, the table **violates 2NF**.

5 Converting to 2NF (Normalization)

To convert the **EMP_PROJ** table into **2NF**, we **remove partial dependencies** by decomposing it into two separate tables.

Step 1: Create Two Separate Relations

1. **EMP (Employee Table - Stores Employee-Specific Data)**
   - Primary Key: `Ssn`
   - Contains attributes **fully dependent** on `Ssn`.

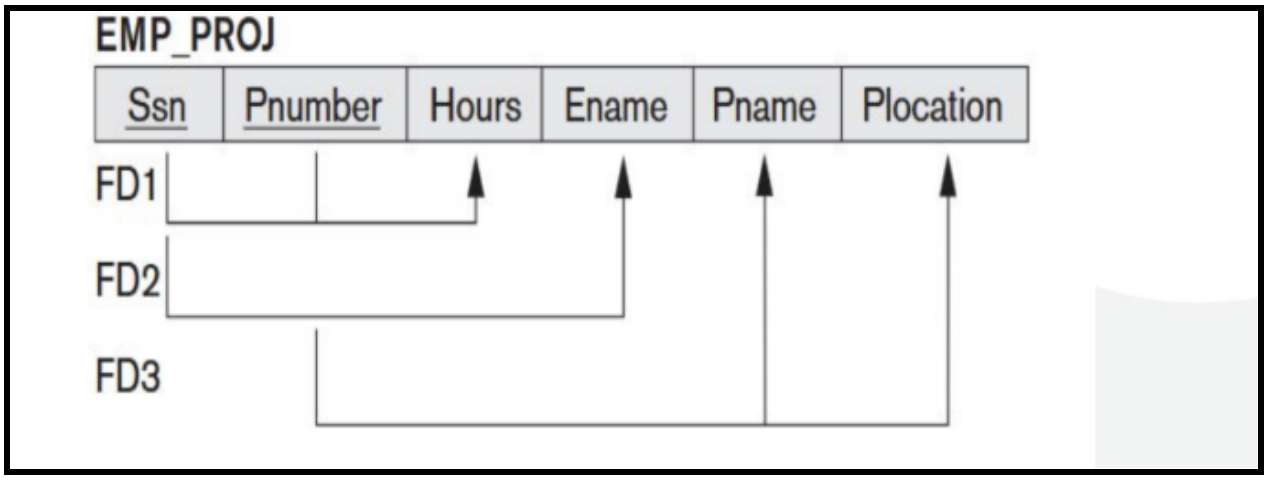
Ssn	Ename
123	Alice
456	Bob

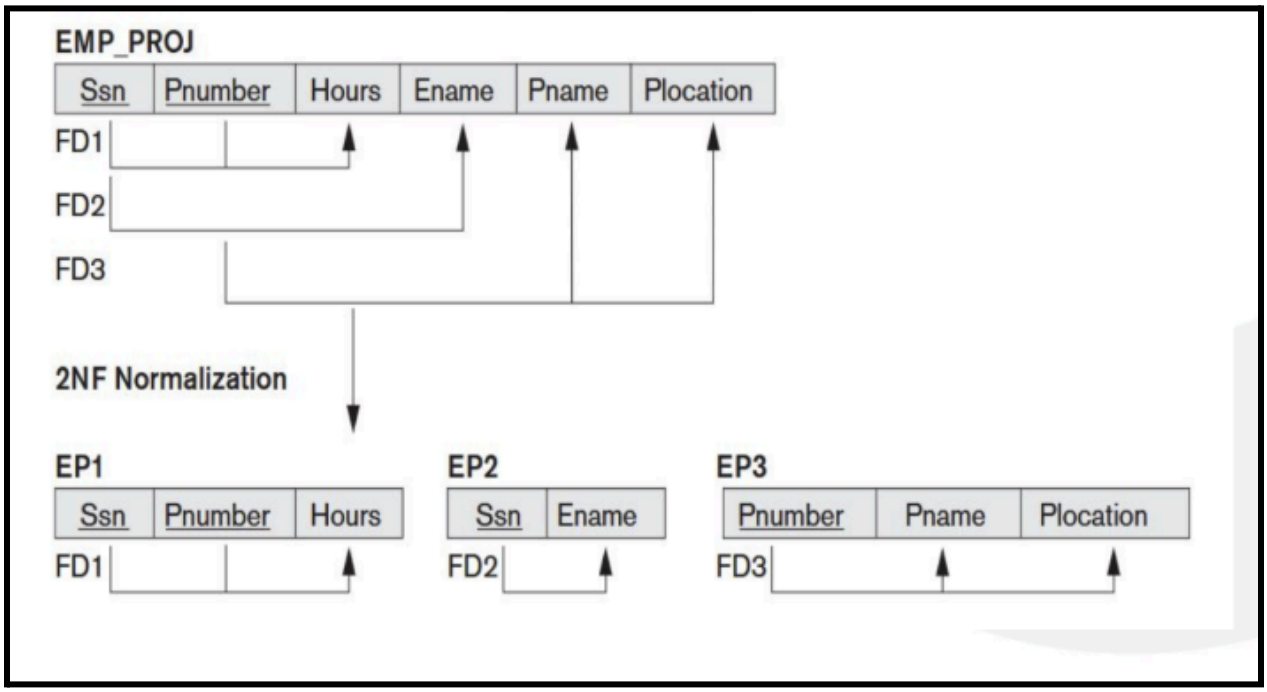
2. **PROJ (Project Table - Stores Project-Specific Data)**
   - Primary Key: `Pnumber`
   - Contains attributes **fully dependent** on `Pnumber`.

Pnumber	Pname	Plocation
1	ProjectA	NY
2	ProjectB	LA

3. **EMP_PROJ_HOURS (Associative Table - Stores Relationship Data)**
   - Primary Key: `{Ssn, Pnumber}`
   - Stores attributes **fully dependent** on `{Ssn, Pnumber}`.

Ssn	Pnumber	Hours
123	1	30
123	2	40
456	2	20





Third normal form (3NF):

Third Normal Form (3NF) is an improvement over 2NF, ensuring that no transitive dependencies exist between non-key attributes and the primary key.

Rules for a relation in 3NF:

1. It is already in 2NF.
2. There are no transitive dependencies, meaning no non-prime attribute should depend on another non-prime attribute.
3. A non-prime attribute (not part of the primary key) should depend only on the primary key.

ex: R(A,B,C,D)

FD: A → B B → C C → D

(i) ⇒ c.key = {A}

(ii) ⇒ prime attr = A

⇒ non prime = B C D.

(iii) * check FDS if NPA → NPA. If yes, not in 3NF.

A → B ✗

B → C ✓ ∴ not in 3NF.

(or)

* check if LHS is s.key (or cand key) ⇒ 3NF //

(or)

* check if RHS has prime attrs ⇒ 3NF //

→ even if 1 NPA is present with P.A in LHS, it is still considered as NPA

any one

ex: R(ABCDEF)

FD: {AB → CDEF, BD → F, A → D}

(i) ⇒ C.K = {A, B}

(ii) ⇒ prime = {A, B} non prime = {C, D, E, F}

(iii) ⇒ 3 conditions

AB → CDEF ✓ (if LHS is P.A, no need to check RHS)

A → D ✓ (〃 〃)

BD → F ✗ (if RHS has one P.A & one NPA; it is completely considered as NPA on LHS).

PA NPA ∴ not 3NF

3 Example of a Relation That is NOT in 3NF

Consider the EMP_DEPT table:

Ssn	Ename	Dnumber	Dname	Dmgr_ssn
123	Alice	5	HR	987
124	Bob	3	IT	654
125	Charlie	5	HR	987

Functional Dependencies (FDs):

1. Ssn → {Ename, Dnumber}
2. Dnumber → {Dname, Dmgr_ssn}

🚫 Why is EMP_DEPT Not in 3NF?

- Dmgr_ssn is **transitively dependent** on `Ssn` through `Dnumber`.
- Since `Dnumber` is **not a candidate key**, the relation **violates 3NF**.

4️⃣ Converting to 3NF (Normalization)

To remove the **transitive dependency**, we **split the relation** into two separate tables.

Step 1: Create Two Separate Relations

1. **EMP (Stores Employee-Specific Data)**
   - Primary Key: `Ssn`
   - Contains attributes **fully dependent** on `Ssn`.

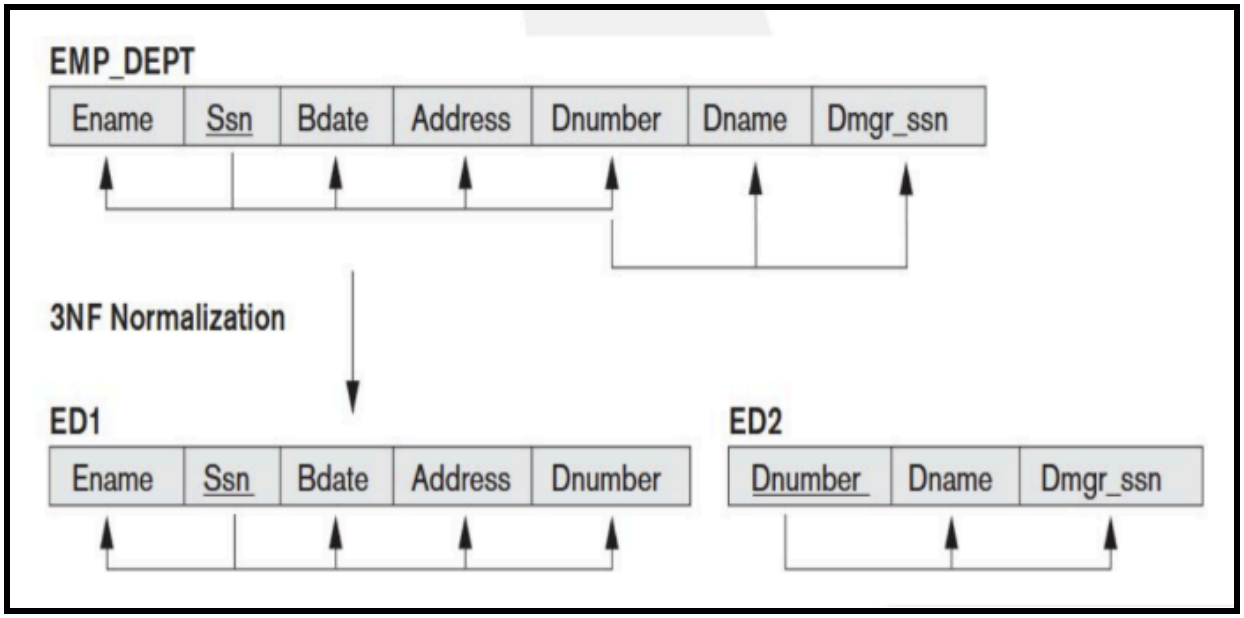
Ssn	Ename	Dnumber
123	Alice	5
124	Bob	3
125	Charlie	5

2. **DEPT (Stores Department-Specific Data)**
   - Primary Key: `Dnumber`
   - Contains attributes **fully dependent** on `Dnumber`.

Dnumber	Dname	Dmgr_ssn
5	HR	987
3	IT	654

✅ **Now in 3NF:**

- EMP contains only **attributes directly dependent on** `Ssn`.
- DEPT contains only **attributes directly dependent on** `Dnumber`.
- **No transitive dependencies remain.**



Boyce-Codd Normal Form (BCNF):

Boyce-Codd Normal Form (BCNF) is a stricter version of 3NF.
Every relation in BCNF is also in 3NF, but not every 3NF relation is in BCNF.
BCNF eliminates certain anomalies that 3NF does not always prevent.

Rules: for BCNF

It is already in 3NF.
For every non-trivial functional dependency (X → Y), X must be a superkey.

4️⃣ Example of a Relation That is NOT in BCNF

Consider the following table **COURSES**:

Course_ID	Instructor	Room
C1	Alice	R101
C2	Bob	R102
C3	Alice	R101

Functional Dependencies (FDs):

1. `Course_ID → Instructor` ✅ (Candidate Key Dependency)
2. `Instructor → Room` ❌ (Issue: Instructor is not a candidate key)

🚫 **Why is this NOT in BCNF?**

- The second FD (`Instructor → Room`) **violates BCNF** because `Instructor` is not a **superkey**.
- This means **the relation still has redundancy**, even though it satisfies **3NF**.

5️⃣ Converting to BCNF (Normalization)

To convert to BCNF, we **split the relation** into two smaller tables.

Step 1: Create Two Separate Relations

1. **COURSE_INSTRUCTOR** (Stores which instructor teaches which course)
   - **Primary Key:** `Course_ID`

Course_ID	Instructor
C1	Alice
C2	Bob
C3	Alice

2. **INSTRUCTOR_ROOM** (Stores which instructor uses which room)
   - **Primary Key:** `Instructor`

Instructor	Room
Alice	R101
Bob	R102

✅ **Now in BCNF:**

- **No functional dependency has a non-superkey determinant.**
- **All anomalies (redundancy) are eliminated.**

Stu_ID	Stu_Branch	Stu_Course	Branch_Number	Stu_Course_No
101	Computer Science & Engineering	DBMS	B_001	201
101	Computer Science & Engineering	Computer Networks	B_001	202
102	Electronics & Communication Engineering	VLSI Technology	B_003	401
102	Electronics & Communication Engineering	Mobile Communication	B_003	402

- Functional dependency for the above tables are:
- Stu_ID -> Stu_Branch
  Stu_Course -> {Branch_Number, Stu_Course_No}
- Candidate Keys of the above table are: **{Stu_ID, Stu_Course}**
- For satisfying this table in BCNF, we have to decompose it into further tables.
- **Stu_Branch Table:**

Stu_ID	Stu_Branch

- **Stu_Course Table:**

Stu_Course	Branch_Number	Stu_Course_No

- **Stu_Enroll Table:**

Stu_ID	Stu_Course_No

PROPERTIES OF RELATIONAL DECOMPOSITIONS

- Dependency Preservation and Property of a Decomposition
- Nonadditive (Lossless) Join Property of a Decomposition

Lossless Property:

Definition: Formally, a decomposition D = {R1,R2,...,Rm} of R has the lossless (nonadditive) join property with respect to the set of dependencies F on R if, for every relation state r of R that satisfies F, the following holds, where * is the NATURAL JOIN of all the relations in D: *(πR1 (r), ..., πRm (r)) = r.

(c) $R = \{Ssn, Ename, Pnumber, Pname, Plocation, Hours\}$ $D = \{R_1, R_2, R_3\}$

$R_1 = EMP = \{Ssn, Ename\}$

$R_2 = PROJ = \{Pnumber, Pname, Plocation\}$

$R_3 = WORKS_ON = \{Ssn, Pnumber, Hours\}$

$F = \{Ssn \rightarrow Ename; Pnumber \rightarrow \{Pname, Plocation\}; \{Ssn, Pnumber\} \rightarrow Hours\}$

	Ssn	Ename	Pnumber	Pname	Plocation	Hours
R_1	a_1	a_2	b_{13}	b_{14}	b_{15}	b_{16}
R_2	b_{21}	b_{22}	a_3	a_4	a_5	b_{26}
R_3	a_1	b_{32}	a_3	b_{34}	b_{35}	a_6

(Original matrix S at start of algorithm)

	Ssn	Ename	Pnumber	Pname	Plocation	Hours
R_1	a_1	a_2	b_{13}	b_{14}	b_{15}	b_{16}
R_2	b_{21}	b_{22}	a_3	a_4	a_5	b_{26}
R_3	a_1	$\cancel{b_{32}}$ a_2	a_3	$\cancel{b_{34}}$ a_4	$\cancel{b_{35}}$ a_5	a_6

(Matrix S after applying the first two functional dependencies; last row is all "a" symbols so we stop)

Step-1:
Construct a table with the decompositions **R1, R2…Rn** v/s the universal relation **R**.

Step-2:
We **apply the given functional dependencies** step by step to **propagate values** across the matrix.

1. **Applying *Ssn → Ename***
   - Since Ssn determines **Ename**, all occurrences of **Ssn** should result in the **same Ename value**.
   - In row $\mathbf{R}_3$, we replace $\mathbf{b}_{32}$ with $\mathbf{a}_2$ (from R_1), since both rows share the same **Ssn** (a_1).
2. **Applying *Pnumber → {Pname, Plocation}***
   - Since **Pnumber** determines both **Pname** and **Plocation**, all occurrences of **Pnumber** should result in **consistent** values for **Pname** and **Plocation**.
   - In row $\mathbf{R}_3$, we replace $\mathbf{b}_{34}$ with $\mathbf{a}_4$ and $\mathbf{b}_{35}$ with $\mathbf{a}_5$, taking values from row $\mathbf{R}_2$, because **Pnumber** = $\mathbf{a}_3$ in both cases.

Step-3:

- When a row consists only of **"a"** values, the decomposition is considered lossless. **We stop after the 2nd FD as we are getting all "a" rows.**
- Since the **last row is now fully defined,** no further updates are needed. Therefore, it is ***LOSSLESS.***
- It **checks whether the decomposition preserves functional dependencies**.
- If **all dependencies are satisfied**, the decomposition **is lossless and valid**.

(a) R = {Ssn, Ename, Pnumber, Pname, Plocation, Hours} $D = \{R_1, R_2\}$
R_1 = EMP_LOCS = {Ename, Plocation}
R_2 = EMP_PROJ1 = {Ssn, Pnumber, Hours, Pname, Plocation}

F = {Ssn → Ename; Pnumber → {Pname, Plocation}; {Ssn, Pnumber} → Hours}

	Ssn	Ename	Pnumber	Pname	Plocation	Hours
R_1	b_{11}	a_2	b_{13}	b_{14}	a_5	b_{16}
R_2	a_1	b_{22}	a_3	a_4	a_5	a_6

(No changes to matrix after applying functional dependencies)

The above decomposition is ***LOSSY***. A **dependency is considered lossy** when the **decomposition of a relation results in data loss** when trying to reconstruct the original relation.
How?
1. In the 1st FD, Ssn column does not have any same "a" rows so we don't need to update Ename.
2. In the 2nd FD, Pnumber does not have any same "a" rows so no changes to be made to Pname and Plocation. Similarly, for the third FD.

Algorithm 16.3. Testing for Nonadditive Join Property

Input: A universal relation R, a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R, and a set F of functional dependencies.

Note: Explanatory comments are given at the end of some of the steps. They follow the format: (* *comment* *).

1. Create an initial matrix S with one row i for each relation R_i in D, and one column j for each attribute A_j in R.
2. Set $S(i, j) := b_{ij}$ for all matrix entries. (* each b_{ij} is a distinct symbol associated with indices (i, j) *).
3. For each row i representing relation schema R_i
   {for each column j representing attribute A_j
      {if (relation R_i includes attribute A_j) then set $S(i, j) := a_j$;};}; (* each a_j is a distinct symbol associated with index (j) *).
4. Repeat the following loop until a *complete loop execution* results in no changes to S
   {for each functional dependency $X \rightarrow Y$ in F
      {for all rows in S *that have the same symbols* in the columns corresponding to attributes in X
         {make the symbols in each column that correspond to an attribute in Y be the same in all these rows as follows: If any of the rows has an a symbol for the column, set the other rows to that *same* a symbol in the column. If no a symbol exists for the attribute in any of the rows, choose one of the b symbols that appears in one of the rows for the attribute and set the other rows to that same b symbol in the column ;} ; } ;};
5. If a row is made up entirely of a symbols, then the decomposition has the nonadditive join property; otherwise, it does not.

Difference Between Lossless and Lossy Decomposition in Databases

Feature	Lossless Decomposition ✅	Lossy Decomposition ❌
Definition	Decomposition where the original relation can be perfectly reconstructed using **natural join (⋈)**.	Decomposition where **data loss or spurious tuples** appear after rejoining.
Data Integrity	**No data is lost** or altered when decomposing and recomposing the relation.	**Data is either lost or corrupted**, causing incorrect query results.
Condition for Validity	The **intersection of decomposed tables contains a candidate key** ensuring no loss.	The intersection **does not contain a candidate key**, leading to loss of dependencies.
Effect on Functional Dependencies (FDs)	**All functional dependencies are preserved**, ensuring consistency.	**Some FDs are lost**, leading to anomalies and inconsistency in data.
Example	✅ **EMP (Emp_ID, Name, Dept_ID)** and **DEPT (Dept_ID, Dept_Name)** → No redundant data and maintains integrity.	❌ **EMP_DEPT (Emp_ID, Name, Dept_ID, Dept_Name)** split into **EMP(Emp_ID, Name)** and **DEPT(Dept_ID, Dept_Name)** → Results in **spurious tuples** when joined back.

Lossless	Lossy
The decompositions R1, R2, R2…Rn for a relation schema R are said to be Lossless if there natural join results the original relation R.	The decompositions R1, R2, R2…Rn for a relation schema R are said to be Lossy if there natural join results into addition of extraneous tuples with the original relation R.
Formally, Let R be a relation and R1, R2, R3 … Rn be it's decomposition, the decomposition is lossless if – R1 ⋈ R2 ⋈ R3 …. ⋈ Rn = R	Formally, Let R be a relation and R1, R2, R3 … Rn be its decomposition, the decomposition is lossy if – R ⊂ R1 ⋈ R2 ⋈ R3 …. ⋈ Rn

Dependency Preservation and Property of a Decomposition:

Definition: Given a set of dependencies F on R, the projection of F on Ri , denoted by ***πRi(F)*** where Ri is a subset of R, is the set of dependencies X → Y in F+ such that the attributes in X ∪ Y are all contained in Ri . Hence, the projection of F on each relation schema Ri in the decomposition D is the set of functional dependencies in F+, the closure of F, such that all their left- and right-hand-side attributes are in Ri . We say that a decomposition D = {R1, R2,..., Rm} of R is dependency-preserving with respect to F if the union of the projections of F on each Ri in D is equivalent to F; that is, ***((πR1 (F))*** ∪ **...** ∪ ***(πRm (F)))+ = F+.***

Q) $R = (A\ B\ C\ D\ E)$ $F = \{A \to B, B \to C, C \to D, D \to A\}$.

$R_1 = (ABC)$ $R_2 = (C\ D\ E)$

$\Rightarrow$ $A^+ = ABCD$ $A \to BC$
$B^+ = BCDA$ $B \to CA$
$C^+ = CDAB$ $C \to AB$
$AB^+ = ABCD$ ~~$AB \to C$~~ dup (duplicate F.D as A alone can derive C)
$AC^+ = X$
$BC^+ = X$

(trick: if A can derive all attrs $\Rightarrow$ sub reln, no need to check for other)

$\therefore F_1 = \{A \to BC, B \to CA, C \to AB\}$

$\Rightarrow$ $C^+ = CDAB$ $C \to D$
$D^+ = CDAB$ $D \to C$
$E^+ = E$
$CD^+ = X$
$CE^+ = CEADB$ ~~$CE \to D$~~ dup
$DE^+ = DEABC$ ~~$DE \to C$~~ dup

(as C & D alone can derive the same)

$F_2 = \{C \to D, D \to C\}$

condition

$\Rightarrow$ $\boxed{F_1 \cup F_2 \equiv F}$ $\Rightarrow$ dep^n preserving

G F

$\Rightarrow$ find $G \supseteq F$ and $F \supseteq G$ (as learnt for equivalence)

$\Rightarrow$ $G = \{A \to BC, B \to CA, C \to AB, C \to D, D \to C\}$

$F = \{$ question $\}$

find if $G \equiv F$

$\therefore$ Yes, dep prev satisfied.

DECOMPOSITION OF RELATIONS TO 2NF AND 3NF

Q) Convert a reln to 2NF form / decompose into 2NF form:

(i) $R(ABCD)$ $F = \{A \to B, B \to C\}$

step 1

$\Rightarrow$ normally check if it is in 2NF:

$ABCD^+ = ABCD$
$ABD^+ = ABCD$
$AD^+ = ABCD$
$\hookrightarrow A^+ = ABC$
$\hookrightarrow D^+ = D$

(to check if AD^+ is a C.K, check if the proper subsets are S.K, if they are then AD^+ is not CK).
(not SK, $\therefore$ It is a C.K)

$\therefore$ CK = AD.
P.A = {A, D}

(only 1 C.K is present as the P.A are not on the RHS of FDs)

$\therefore$ the 1st FD is violating 2NF rules as there is partial dep^n
$A \to B$

step 2

$\Rightarrow$ find closure of L.H.S of F.D creating a problem & make sub-relns:

$A^+ = ABC$

$R_1 = \{A, B, C\}$ $\quad R_2 = \{D\}$ } we have R_1 with closure ele. of 'R' R_2 has remaining from 'R'.

But it is not a proper decomposition & it shd be dependency preserving & lossless. But there are no common attbs b/w R_1 & R_2. So, it is Lossy decom.

Step 3 add a common attb. The common attb should be a S.K of atleast one sub-relⁿ, then it is said to be Lossless decom.

$R_1 = \{A, B, C\}$ $\quad R_2 = \{A, D\}$ → $A^+ = ABC$ (A is SK of R_1 as all can be determined)

∴ we have decomposed into R_1 & R_2 //.

Step 4 find F.D of R_1 & R_2:

$R_1 = \{A, B, C\}$

$A^+ = ABC$ $\quad AB^+ = X$ no need to check

$B^+ = BC$ $\quad AC^+ = X$

$C^+ = C$ $\quad BC^+ = BC$

∴ $\{A \to BC, B \to C\} = F_1$ //

$R_2 = \{A, D\}$ → (since only 2 attbs are present R_2 is already in BCNF. LHS has S.K in the F.D)

$A^+ = ABC$ (not a part of R_2)

$D^+ = D$

∴ $\{\phi\} = F_2$ //

∴ CK $= \{A\}$

∴ P.A $= \{A\}$ N.PA $= \{\phi\}$

Check ∴ 2NF ✓ //

(If C.K is single then there will be no chance of P.Dep as there will not be any proper subsets of A. Hence, no need to check RHS)

Step 5 Check dependency preservation: $F_1 \cup F_2 = F$ → of question given

F_1 of R_1, F_2 of R_2

∴ $F_1 \cup F_2 = \{\phi\} \cup \{A \to BC, B \to C\}$

$= \{A \to BC, B \to C\}$

∴ exists in F //.

(ii) $R(ABCD)$ $F = \{A \to B, C \to D\}$

⇒ $ABCD^+ = ABCD$

$AC^+ = ACBD$

↳ $A^+ = AB$ X

↳ $C^+ = CD$ X

∴ CK = AC

∴ P.A = $\{A, C\}$ ∴ only 1 C.K //.

⇒ Both $A \to B$ & $C \to D$ have Partial dep.

⇒ $A^+ = AB$ $\quad C^+ = CD$ $\quad R_3 = \{A, C\}$ → no remaining attbs → do this

$R_1 = \{A, B\}$ $\quad R_2 = \{C, D\}$

(as to lossless, S.K of sub relⁿ should be common. So, take A i.e SK of R_1. add A to R_3. Now check in R_2, C is S.K. So put C in R_3.

∴ now decom^n is lossless//

⇒ now find if it is dep preserving or not after finding FDs:

$R_1 = \{A, B\}$
$A^+ = AB$
$B^+ = B$

$R_2 = \{C, D\}$
$C^+ = CD$
$D^+ = D$

$R_3 = \{A, C\}$ → By default all in BCNF But, you can do it manually
$A^+ = AB$
$C^+ = CD$

$F_1 = \{A \to B\}$ //

$F_2 = \{C \to D\}$ //

$F_3 = \{\phi\}$

C.K = A
2NF (& BCNF (LHS = SK))

C.K = C
2NF (& BCNF (LHS = SK))

⇒ $F_1 \cup F_2 \cup F_3 = F$

$\{A \to B\} \cup \{C \to D\} = F$ // ∴ Dep prev satisfied//.

Q) Convert 2NF to 3NF / decompose into 3NF:

(i) $R(ABCD)$ $F\{A \to B, B \to C, C \to D\}$

⇒ Check for 3NF normally:

$ABCD^+ = ABCD$
$A^+ = ABCD$ (no proper subsets)//
∴ CK = A
∴ P.A = {A} (only 1 CK present)//

⇒ $B \to C$, $C \to D$ are creating prob due to transitive dep^n.

⇒ $B^+ = BCD$
$R_1 = \{B, C, D\}$
$B^+ = BCD$
$C^+ = CD$
$D^+ = D$
~~BC~~
~~BD~~ }×
~~CD~~ = CD

$F_1 = \{B \to CD, C \to D\}$ //

$\boxed{C.K = B}$

$C^+ = CD$
$R_2 = \{C, D\}$
$C^+ = CD$
$D^+ = D$

$F_2 = \{C \to D\}$ //

$R_3 = \{A, B\}$ → why not C?
$A^+ = AB$
$B^+ = B$

$F_3 = \{A \to B\}$ //

{R_2 & R_3 are in BCNF already}

∴ $B \to CD$ does not have transitive dep but $C \to D$ does have//.

∴ decompose R_1 again//.

$\Rightarrow R_1 = \{B\ C\ D\}$

$F = \{B \to CD,\ C \to D\}$

Problem $\Rightarrow$ T.Dep^n

$C^+ = CD$

$R_{11} = \{C, D\}$

$F_{11} = \{C \to D\}$ //

$\boxed{C \cdot K = C}$

$\therefore$ no transitive

$\therefore$ 3NF & BCNF //

$R_{12} = \{B, C\}$ $\rightarrow$ common for lossless

$F_{12} = \{B \to C\}$ //

$\boxed{C \cdot K = B}$

$\therefore$ No trans

$\therefore$ 3NF & BCNF //

$\therefore$ Totally we have $\underset{\text{discard}}{R_1}$, R_2, $\underset{\text{discard as } \{C,D\} \text{ already in } R_2}{R_3}$, R_{11}, R_{12} $\Longrightarrow$ $R_{12}(B,C)$, $R_2(C,D)$, $R_3(A,B)$

[also satisfies lossless & D.pres properties]

Q. Consider the relation scheme *R = (A,B,C,D,E,F,G,H,I,J,K,L)* with the set of functional dependencies *FD = { {AB}->{C} , {B}->{F}, {F}->{G,H} , {D}->{E,I,J} , {B}->{K,L} , {K}->{L} }. Determine the key of R and decompose R into 2NF and 3NF relations.

Q. Given the relation schema *R = (A, B, C, D, E, F, G, H)* and the following set of functional dependencies: *F = { A→ B , ABCD→ E , EF → G , EF → H , ACDF → EG }*
Decompose the given relation into 3NF. Give proper justification.